



DATABASE SYSTEMS PROJECT REPORT

**AI-Powered Campus Recruitment & Applicant Tracking
System**

1. Group Members

Member 01: Samiya Khan (24K-0768)

- SQL: Accounts, Companies, Students, Recruiters tables, procedures, triggers, and Active_Jobs_View
- Backend: Authentication, student modules, and Resume Keyword Matching Agent
- Frontend: Login, registration, dashboard, and profile pages

Member 02: Iqra Ashraf (24K-0952)

- SQL: Skills, Job_Posts, Job_Required_Skills tables, procedures, triggers, and views
- Backend: Job management, recommendations module, and Job Recommendation Agent
- Frontend: Student jobs, applications, and recommendations pages

Member 03: Aseelah Masood Khan (24K-0881)

- SQL: Student_Skills, Applications, Resumes, AuditLog tables, indexes, constraints, and testing
- Backend: Applications module and Utility-Based Ranking Agent
- Frontend: Recruiter dashboard and management pages

2. Project Title

AI-Powered Campus Recruitment & Applicant Tracking System

3. Project Description

The AI-Powered Campus Recruitment & Applicant Tracking System is a web-based platform developed to simplify and automate the recruitment process between students and recruiters. The system provides a centralized environment where students can create profiles, upload resumes, browse job opportunities, and apply for positions. Recruiters can create job postings, manage applications, and evaluate candidates through an organized dashboard.

The project uses a PostgreSQL relational database with a normalized schema to maintain data consistency and integrity. The backend is developed using FastAPI, while the frontend is built using HTML, CSS, and JavaScript. The system also includes AI-based modules for resume skill extraction, applicant ranking, and personalized job recommendations to improve the efficiency of the recruitment process.

4. Project Requirements

4.1 Functional Requirements

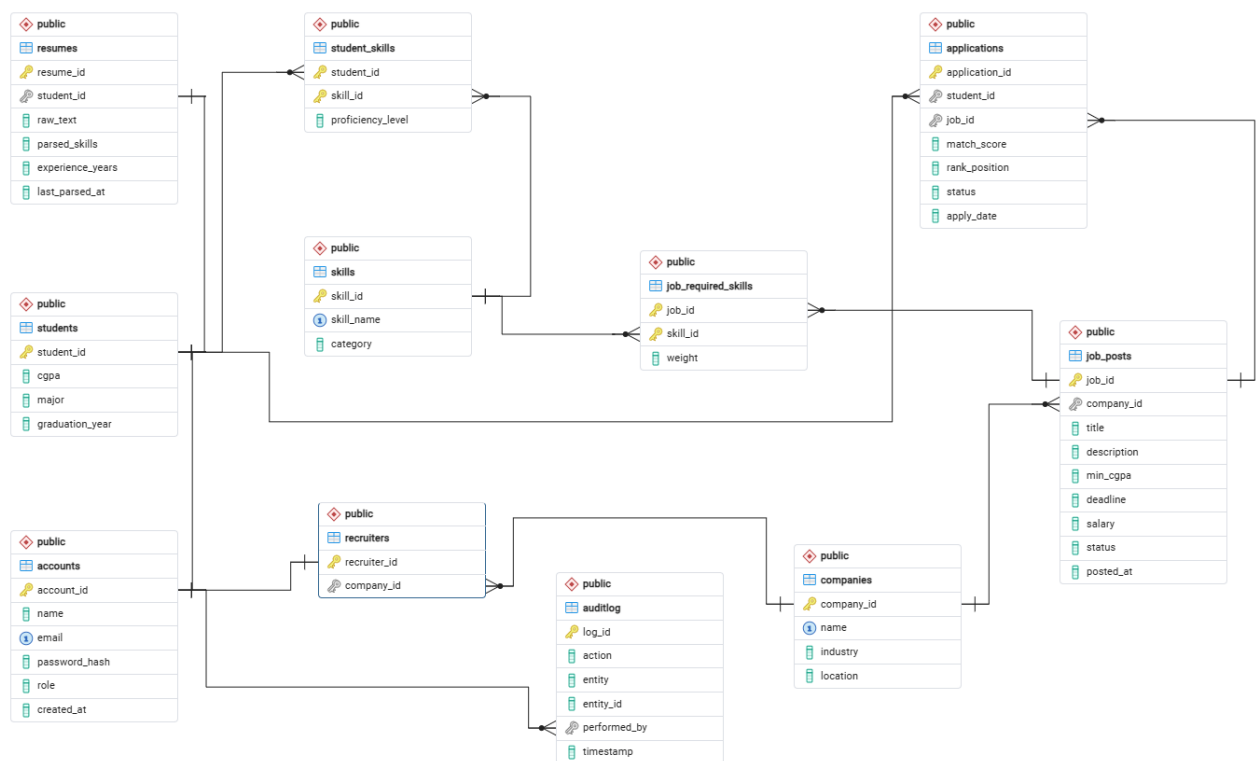
- Student and recruiter registration and login system
- Profile management for students and recruiters
- Job posting and application management
- Resume upload and automatic skill extraction
- AI-based candidate ranking system
- Personalized job recommendation system
- Application tracking and status management
- Role-based access control for students and recruiters

4.2 Database Requirements

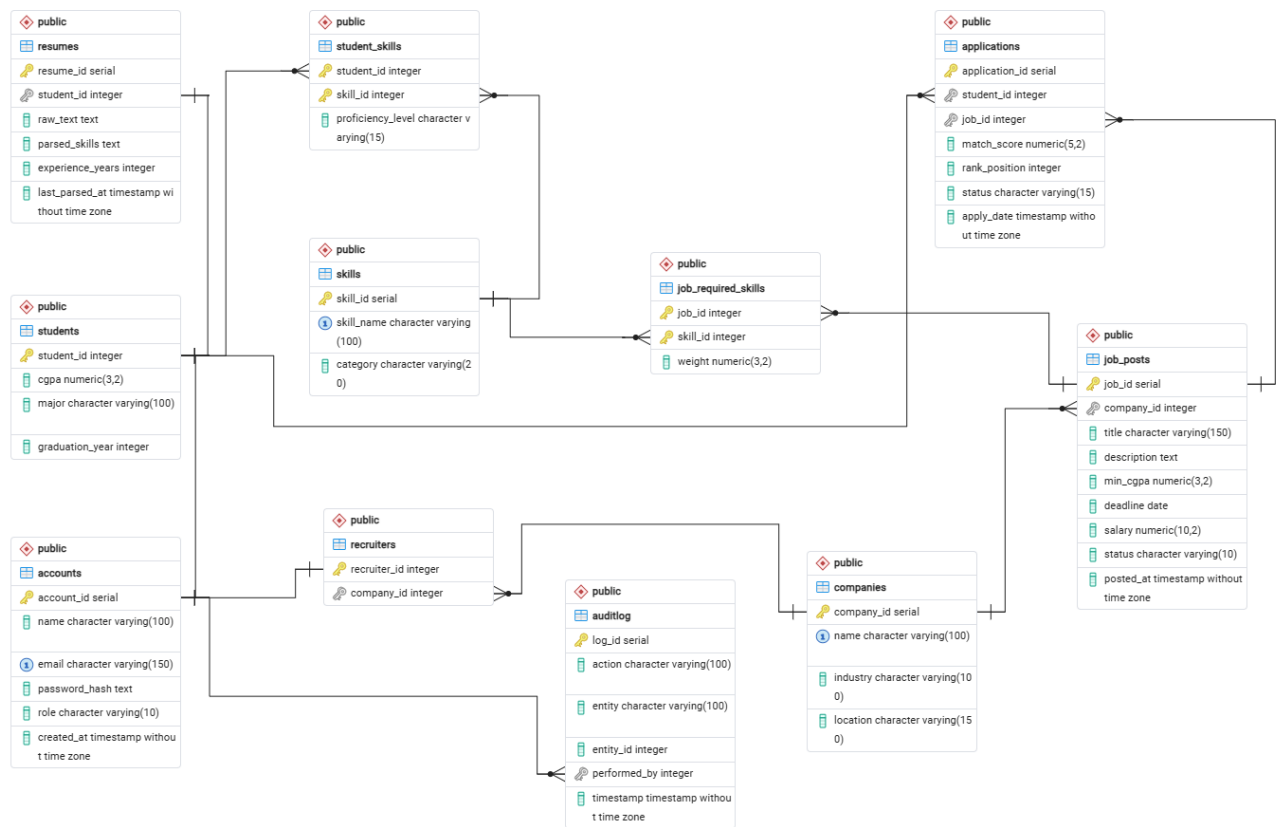
- Normalized relational database schema
- Use of primary keys and foreign keys
- Implementation of constraints and validation checks
- Stored procedures for application handling and recommendations
- Triggers for automated database actions
- Views and indexes for optimized queries and reporting

5. ER Diagram

Without Data Types



With Data Types



6. Normalized Schema

```

Accounts(
    account_id PK
    name NOT NULL
    email NOT NULL UNIQUE
    password_hash NOT NULL
    role NOT NULL CHECK ('student', 'recruiter')
    created_at NOT NULL
)
    
```

```

Companies(
    company_id PK
    name NOT NULL UNIQUE
    industry
    location
)
    
```

```
Students(
    student_id PK, FK → Accounts(account_id)
    cgpa CHECK (0.0–4.0)
    major
    graduation_year
)
```

```
Recruiters(
    recruiter_id PK, FK → Accounts(account_id)
    company_id NOT NULL FK → Companies(company_id)
)
```

```
Skills(
    skill_id PK
    skill_name NOT NULL UNIQUE
    category NOT NULL CHECK ('technical', 'soft', 'domain')
)
```

```
Job_Posts(
    job_id PK
    company_id NOT NULL FK → Companies(company_id)
    title NOT NULL
    description
    min_cgpa NOT NULL CHECK (0.0–4.0)
    deadline NOT NULL
    salary NOT NULL
    status NOT NULL CHECK ('active', 'closed', 'draft')
    posted_at NOT NULL
)
```

```
Student_Skills(
    student_id PK, FK → Students(student_id)
    skill_id PK, FK → Skills(skill_id)
    proficiency_level NOT NULL CHECK ('beginner', 'intermediate', 'advanced')
)
```

```
Job_Required_Skills(
    job_id PK, FK → Job_Posts(job_id)
    skill_id PK, FK → Skills(skill_id)
    weight CHECK (0.5, 1.0)
)
```

```
Applications(
```

```
application_id PK
student_id NOT NULL FK → Students(student_id)
job_id NOT NULL FK → Job_Posts(job_id)
match_score
rank_position
status NOT NULL CHECK ('submitted', 'shortlisted', 'rejected', 'accepted')
apply_date NOT NULL
UNIQUE (student_id, job_id)
)
```

```
Resumes(
  resume_id PK
  student_id NOT NULL UNIQUE FK → Students(student_id)
  raw_text
  parsed_skills
  experience_years
  last_parsed_at
)
```

```
AuditLog(
  log_id PK
  action
  entity
  entity_id
  performed_by FK → Accounts(account_id)
  timestamp NOT NULL
)
```